

Day 6 Assignment

Building a Data Processing API with Pandas, NumPy and FastAPI

John Michael Bilbao
Techstart
August 14, 2026

Introduction

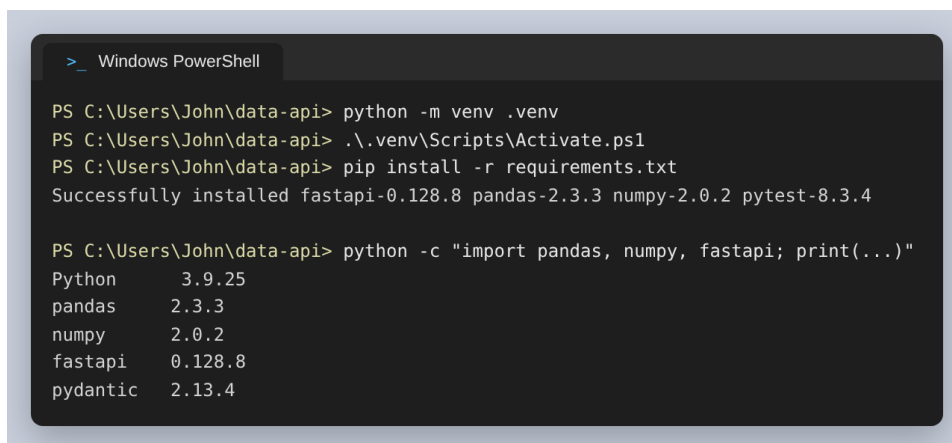
This assignment asks for a FastAPI service that does data processing work with Pandas and NumPy, tested with curl and documented with screenshots.

The dataset is the Iris measurements, 150 rows and five columns, which ships with the project as a CSV. POST /load_data reads it into a Pandas DataFrame that is held in memory, and the other seven endpoints work on that DataFrame. Pandas does the loading, describing, filtering and grouping; NumPy does the statistics and the correlation matrix.

The three endpoints the assignment names are all there, and I added five more, because /load_data and /describe_data on their own do not really exercise either library. Everything is run from PowerShell on Windows.

Step 1: Setting up the environment

A virtual environment, then the four packages. FastAPI is installed with the standard extras, which is what brings in Uvicorn and the fastapi command line tool.



```
> Windows PowerShell

PS C:\Users\John\data-api> python -m venv .venv
PS C:\Users\John\data-api> .\.venv\Scripts\Activate.ps1
PS C:\Users\John\data-api> pip install -r requirements.txt
Successfully installed fastapi-0.128.8 pandas-2.3.3 numpy-2.0.2 pytest-8.3.4

PS C:\Users\John\data-api> python -c "import pandas, numpy, fastapi; print(...)"
Python      3.9.25
pandas      2.3.3
numpy       2.0.2
fastapi     0.128.8
pydantic    2.13.4
```

Figure 1: The environment, and the versions in it

Before writing any endpoint I looked at the data in a plain Python session, because the shape of the API follows from the shape of the dataset. Four float columns and one text column is what leads to /stats refusing text columns and /filter_data needing to convert its value before comparing.

```
>_ Windows PowerShell

PS C:\Users\John\data-api> python -c "import pandas as pd; ..."
shape      (150, 5)
columns     ['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'species']
nulls       0

   sepal_length  sepal_width  petal_length  petal_width  species
0           5.1           3.5           1.4           0.2   setosa
1           4.9           3.0           1.4           0.2   setosa
2           4.7           3.2           1.3           0.2   setosa
3           4.6           3.1           1.5           0.2   setosa
4           5.0           3.6           1.4           0.2   setosa

species
setosa      50
versicolor  50
virginica   50
```

Figure 2: The dataset: 150 rows, four numeric columns, one text column, no missing values

The assignment asks for the project to be started with the fastapi command line tool, so the server runs with fastapi dev. It finds the app object itself, turns on reloading, and prints where the service and its documentation are:

```
>_ Windows PowerShell - fastapi dev

PS C:\Users\John\data-api> fastapi dev data_api\main.py

FastAPI   Starting development server []

          Searching for package file structure from directories with
          __init__.py files
          Importing from C:\Users\John\data-api

module    [] data_api
          |__ [] __init__.py
          |__ [] main.py

code      Importing the FastAPI app object from the module with the following
          code:

          from data_api.main import app

app        Using import string: data_api.main:app

server    Server started at http://127.0.0.1:8001
server    Documentation at http://127.0.0.1:8001/docs

tip       Running in development mode, for production use: fastapi run

Logs:

INFO      Will watch for changes in these directories:
          ['C:\Users\John\data-api']
INFO      Uvicorn running on http://127.0.0.1:8001 (Press CTRL+C to quit)
INFO      Started reloader process [21] using WatchFiles
INFO      Started server process [30]
INFO      Waiting for application startup.
INFO      Application startup complete.
```

Figure 3: fastapi dev starting the development server

Step 2: Designing the API endpoints

Eight endpoints. The three in the assignment, four more that do actual processing, and a health check:

Endpoint	What it does	Library
POST /load_data	Read the CSV into a DataFrame and report its shape, dtypes, memory use and first five rows	Pandas
GET /describe_data	describe(include='all'), so text columns are summarised too	Pandas
GET /filter_data	Filter rows with one of seven comparisons	Pandas
GET /columns	Per-column dtype, null count and distinct count	Pandas
GET /stats/{column}	Mean, median, standard deviation, quartiles and Tukey outlier fences	NumPy
GET /group_by	Group on one column, aggregate another	Pandas

Endpoint	What it does	Library
GET /correlation	Pearson matrix over the numeric columns	NumPy
GET /health	Whether the service is up and data is loaded	—

The design decision that shaped everything else is that the DataFrame is state. It has to be loaded before anything can read it, so every read returns 409 with a message naming the endpoint to call first, rather than an empty result that looks like an answer.

The other decision is that every argument goes in the query string. Nothing takes a request body, which keeps the curl commands short enough to fit in a screenshot and means each endpoint is a URL that can be pasted into a browser.

The failures are three codes: 409 when nothing is loaded, 404 for a column that does not exist or is the wrong type, and 400 when the arguments cannot be applied to the data. All of them return the same two fields:

```
{
  "error": "data_not_loaded",
  "message": "No dataset is loaded. Call POST /load_data first."
}
```

Step 3: Implementing the endpoints

The endpoints themselves are thin. Each one gets the DataFrame, calls a function in `processing.py` and wraps the result in a response model. `/filter_data` is the most involved, because it is the one with real parameters:

```
• main.py  processing.py  schemas.py

154 @app.get("/filter_data", response_model=FilterResult, tags=["data"],
155         summary="Filter rows on a condition",
156         description="Builds a boolean mask and returns the rows that match. "
157                     "The value arrives as text and is converted to a number "
158                     "when the column is numeric, so `value=5` and `value=5.0` "
159                     "behave the same on a float column.",
160         responses={409: ERRORS[409], 404: ERRORS[404], 400: ERRORS[400]})
161 def filter_data(
162     column: str = Query(description="Column to test.",
163                         examples=["petal_length"]),
164     op: Literal["eq", "ne", "gt", "gte", "lt", "lte", "contains"] = Query(
165         "eq", description="Comparison to apply."),
166     value: str = Query(description="Value to compare against.",
167                       examples=["5.0"]),
168     limit: int = Query(10, ge=1, le=150,
169                      description="Most rows to return."),
170 ) -> FilterResult:
171     frame = state.frame()
172     matched = proc.filter_rows(frame, column, op, value)
173     head = matched.head(limit)
174     return FilterResult(
175         column=column, op=op, value=value,
176         matched=int(len(matched)), total=int(len(frame)),
177         returned=int(len(head)), limit=limit,
178         rows=proc.records(head),
179     )

data_api/main.py Python Spaces: 4 UTF-8
```

Figure 4: The `/filter_data` endpoint. Declaring `op` as a `Literal` is what makes FastAPI reject an unknown operator before the function runs.

Declaring `op` as a `Literal` rather than a string is worth pointing out. It gives FastAPI the list of valid operators, so a bad one is rejected with a 422 before the endpoint body executes, and it turns into a dropdown in the generated documentation. I did not have to write that validation.

The NumPy work is in `processing.py`. Pandas would give most of these figures directly, but the outlier bounds are easier to express as arithmetic on percentiles, and the assignment asks for NumPy:

```
• processing.py  main.py  schemas.py

148 def numeric_stats(frame: pd.DataFrame, column: str) -> Dict[str, Any]:
149     """Statistics for one numeric column, computed with NumPy.
150
151     Pandas would give most of these directly; NumPy is used here because the
152     assignment asks for it, and because the outlier bounds are easier to express
153     as arithmetic on percentiles than as DataFrame operations.
154     """
155     require_numeric(frame, column)
156     values = frame[column].to_numpy(dtype=float)
157     values = values[np.isfinite(values)]
158     if values.size == 0:
159         raise ColumnError(f"Column '{column}' has no usable values.")
160
161     q1, q2, q3 = np.percentile(values, [25, 50, 75])
162     iqr = q3 - q1
163     low, high = q1 - 1.5 * iqr, q3 + 1.5 * iqr
164     outliers = values[(values < low) | (values > high)]
165
166     return {
167         "column": column,
168         "count": int(values.size),
169         "mean": _clean(np.mean(values)),
170         "median": _clean(q2),
171         # ddof=1 is the sample standard deviation, which is what pandas reports.
172         "std": _clean(np.std(values, ddof=1)),
173         "variance": _clean(np.var(values, ddof=1)),
174         "min": _clean(np.min(values)),
175         "max": _clean(np.max(values)),
176         "range": _clean(np.ptp(values)),
177         "q1": _clean(q1),
178         "q3": _clean(q3),
179         "iqr": _clean(iqr),
180         "outlier_bounds": {"lower": _clean(low), "upper": _clean(high)},
181         "outliers": [_clean(v) for v in np.sort(outliers)],
182     }
```

Figure 5: The statistics, computed on the column as a float array

Two details in there caused real problems. NumPy scalars are not JSON serialisable, so every value goes through a helper that converts them to Python numbers. And NaN is not valid JSON at all, so any non-finite value becomes null; that is why describe() comes back with nulls where a statistic does not apply to a text column instead of failing to encode.

Step 4: Testing the API with curl

Every endpoint below was called with curl.exe against the running server. The -i flag prints the status line and the headers as well as the body.

First, the health check before and after loading. This is the state the rest of the API depends on:

```
> _ curl - /health

PS C:\Users\John\data-api> curl.exe -i http://127.0.0.1:8000/health
HTTP/1.1 200 OK
date: Fri, 14 Aug 2026 01:37:51 GMT
server: uvicorn
content-length: 47
content-type: application/json

{
  "status": "ok",
  "data_loaded": false,
  "rows": null
}

PS C:\Users\John\data-api> curl.exe -i http://127.0.0.1:8000/health
HTTP/1.1 200 OK
date: Fri, 14 Aug 2026 01:37:51 GMT
server: uvicorn
content-length: 45
content-type: application/json

{
  "status": "ok",
  "data_loaded": true,
  "rows": 150
}
```

Figure 6: GET /health, before and after the dataset is loaded

Loading the data. The response is the DataFrame's shape, the dtypes Pandas inferred, how much memory it occupies and the first five rows:

```

> _ curl - POST /load_data

PS C:\Users\John\data-api> curl.exe -i -X POST http://127.0.0.1:8000/load_data
HTTP/1.1 200 OK
date: Fri, 14 Aug 2026 01:37:51 GMT
server: uvicorn
content-length: 810
content-type: application/json

{
  "source": "data/iris.csv",
  "loaded_at": "2026-08-14T01:37:51+00:00",
  "rows": 150,
  "columns": 5,
  "column_names": [
    "sepal_length",
    "sepal_width",
    "petal_length",
    "petal_width",
    "species"
  ],
  "dtypes": {
    "sepal_length": "float64",
    "sepal_width": "float64",
    "petal_length": "float64",
    "petal_width": "float64",
    "species": "object"
  },
  "memory_bytes": 14728,
  "preview": [
    {
      "sepal_length": 5.1,
      "sepal_width": 3.5,
      "petal_length": 1.4,
      "petal_width": 0.2,
      "species": "setosa"
    },
    {
      "sepal_length": 4.9,
      "sepal_width": 3.0,
      "petal_length": 1.4,
      "petal_width": 0.2,
      "species": "setosa"
    },
    {
      "sepal_length": 4.7,
      "sepal_width": 3.2,
      "petal_length": 1.3,
      "petal_width": 0.2,
      "species": "setosa"
    },
    "... 2 more rows not shown"
  ]
}

```

Figure 7: POST /load_data

The per-column view, which is what tells a client which columns it can ask for statistics on. species is the only one with numeric false, and its three distinct values are the three species:

```
> curl - /columns

PS C:\Users\John\data-api> curl.exe -i http://127.0.0.1:8000/columns
HTTP/1.1 200 OK
date: Fri, 14 Aug 2026 01:37:51 GMT
server: uvicorn
content-length: 486
content-type: application/json

{
  "rows": 150,
  "columns": [
    {
      "name": "sepal_length",
      "dtype": "float64",
      "numeric": true,
      "non_null": 150,
      "nulls": 0,
      "unique": 35
    },
    {
      "name": "sepal_width",
      "dtype": "float64",
      "numeric": true,
      "non_null": 150,
      "nulls": 0,
      "unique": 23
    },
    {
      "name": "petal_length",
      "dtype": "float64",
      "numeric": true,
      "non_null": 150,
      "nulls": 0,
      "unique": 43
    },
    {
      "name": "petal_width",
      "dtype": "float64",
      "numeric": true,
      "non_null": 150,
      "nulls": 0,
      "unique": 22
    },
    {
      "name": "species",
      "dtype": "object",
      "numeric": false,
      "non_null": 150,
      "nulls": 0,
      "unique": 3
    }
  ]
}
```

Figure 8: GET /columns

describe() across all five columns. Two columns are omitted from the screenshot to keep it on one page; the endpoint returns all five:

```
> curl - /describe_data

PS C:\Users\John\data-api> curl.exe -i http://127.0.0.1:8000/describe_data
HTTP/1.1 200 OK
date: Fri, 14 Aug 2026 01:37:51 GMT
server: uvicorn
content-length: 843
content-type: application/json

{
  "rows": 150,
  "statistics": [
    "count",
    "unique",
    "top",
    "freq",
    "mean",
    "std",
    "min",
    "25%",
    "50%",
    "75%",
    "max"
  ],
  "describe": {
    "sepal_length": {
      "count": 150.0,
      "unique": null,
      "top": null,
      "freq": null,
      "mean": 5.843333,
      "std": 0.828066,
      "min": 4.3,
      "25%": 5.1,
      "50%": 5.8,
      "75%": 6.4,
      "max": 7.9
    },
    "petal_length": {
      "count": 150.0,
      "unique": null,
      "top": null,
      "freq": null,
      "mean": 3.758,
      "std": 1.765298,
      "min": 1.0,
      "25%": 1.6,
      "50%": 4.35,
      "75%": 5.1,
      "max": 6.9
    },
    "species": {
      "count": 150,
      "unique": 3,
      "top": "setosa",
      "freq": 50,
      "mean": null,
      "std": null,
      "min": null,
      "25%": null,
      "50%": null,
      "75%": null,
      "max": null
    },
    "... 2 more columns not shown": "sepal_width, petal_width"
  }
}
```

Figure 9: GET /describe_data

Filtering on the text column. 50 of the 150 rows are setosa, which is what the dataset should give:

```
> _ curl -/filter_data (text)

PS C:\Users\John\data-api> curl.exe -i "http://127.0.0.1:8000/filter_data?column=species&op=eq&value=setosa&limit=3"
HTTP/1.1 200 OK
date: Fri, 14 Aug 2026 01:37:51 GMT
server: uvicorn
content-length: 389
content-type: application/json

{
  "column": "species",
  "op": "eq",
  "value": "setosa",
  "matched": 50,
  "total": 150,
  "returned": 3,
  "limit": 3,
  "rows": [
    {
      "sepal_length": 5.1,
      "sepal_width": 3.5,
      "petal_length": 1.4,
      "petal_width": 0.2,
      "species": "setosa"
    },
    {
      "sepal_length": 4.9,
      "sepal_width": 3.0,
      "petal_length": 1.4,
      "petal_width": 0.2,
      "species": "setosa"
    },
    {
      "sepal_length": 4.7,
      "sepal_width": 3.2,
      "petal_length": 1.3,
      "petal_width": 0.2,
      "species": "setosa"
    }
  ]
}
```

Figure 10: GET /filter_data with an equality match on species

Filtering on a numeric column. 42 rows have a petal length above 5.0, and matched reports the true total while limit controls how many rows come back:

```
> _ curl -/filter_data (numeric)

PS C:\Users\John\data-api> curl.exe -i "http://127.0.0.1:8000/filter_data?column=petal_length&op=gt&value=5.0&limit=3"
HTTP/1.1 200 OK
date: Fri, 14 Aug 2026 01:37:51 GMT
server: uvicorn
content-length: 401
content-type: application/json

{
  "column": "petal_length",
  "op": "gt",
  "value": "5.0",
  "matched": 42,
  "total": 150,
  "returned": 3,
  "limit": 3,
  "rows": [
    {
      "sepal_length": 6.0,
      "sepal_width": 2.7,
      "petal_length": 5.1,
      "petal_width": 1.6,
      "species": "versicolor"
    },
    {
      "sepal_length": 6.3,
      "sepal_width": 3.3,
      "petal_length": 6.0,
      "petal_width": 2.5,
      "species": "virginica"
    },
    {
      "sepal_length": 5.8,
      "sepal_width": 2.7,
      "petal_length": 5.1,
      "petal_width": 1.9,
      "species": "virginica"
    }
  ]
}
```

Figure 11: GET /filter_data with a greater-than comparison

The contains comparison, which is case-insensitive, so VIRGIN in upper case still finds all 50 virginica rows:

```
> _ curl - /filter_data (contains)

PS C:\Users\John\data-api> curl.exe -i "http://127.0.0.1:8000/filter_data?column=species&op=contains&value=VIRGIN&limit=2"
HTTP/1.1 200 OK
date: Fri, 14 Aug 2026 01:37:51 GMT
server: uvicorn
content-length: 306
content-type: application/json

{
  "column": "species",
  "op": "contains",
  "value": "VIRGIN",
  "matched": 50,
  "total": 150,
  "returned": 2,
  "limit": 2,
  "rows": [
    {
      "sepal_length": 6.3,
      "sepal_width": 3.3,
      "petal_length": 6.0,
      "petal_width": 2.5,
      "species": "virginica"
    },
    {
      "sepal_length": 5.8,
      "sepal_width": 2.7,
      "petal_length": 5.1,
      "petal_width": 1.9,
      "species": "virginica"
    }
  ]
}
```

Figure 12: GET /filter_data with a partial, case-insensitive match

The NumPy statistics for petal length. The mean of 3.758 and the median of 4.35 are far apart, which is the two-cluster shape of this dataset showing up in a single column:

```
> curl - /stats/petal_length

PS C:\Users\John\data-api> curl.exe -i http://127.0.0.1:8000/stats/petal_length
HTTP/1.1 200 OK
date: Fri, 14 Aug 2026 01:37:51 GMT
server: uvicorn
content-length: 220
content-type: application/json

{
  "column": "petal_length",
  "count": 150,
  "mean": 3.758,
  "median": 4.35,
  "std": 1.765298,
  "variance": 3.116278,
  "min": 1.0,
  "max": 6.9,
  "range": 5.9,
  "q1": 1.6,
  "q3": 5.1,
  "iqr": 3.5,
  "outlier_bounds": {
    "lower": -3.65,
    "upper": 10.35
  },
  "outliers": []
}
```

Figure 13: GET /stats/petal_length

Sepal width is the column with outliers. The fences come out at 2.05 and 4.05, and four measurements fall outside them:

```
> curl - /stats/sepal_width

PS C:\Users\John\data-api> curl.exe -i http://127.0.0.1:8000/stats/sepal_width
HTTP/1.1 200 OK
date: Fri, 14 Aug 2026 01:37:51 GMT
server: uvicorn
content-length: 234
content-type: application/json

{
  "column": "sepal_width",
  "count": 150,
  "mean": 3.057333,
  "median": 3.0,
  "std": 0.435866,
  "variance": 0.189979,
  "min": 2.0,
  "max": 4.4,
  "range": 2.4,
  "q1": 2.8,
  "q3": 3.3,
  "iqr": 0.5,
  "outlier_bounds": {
    "lower": 2.05,
    "upper": 4.05
  },
  "outliers": [
    2.0,
    4.1,
    4.2,
    4.4
  ]
}
```

Figure 14: GET /stats/sepal_width, where the Tukey fences catch four values

Grouping. Mean petal length per species separates the three cleanly, 1.462 against 4.26 against 5.552, and the second call shows a different aggregation on a different column:

```
>_ curl - /group_by

PS C:\Users\John\data-api> curl.exe -i "http://127.0.0.1:8000/group_by?by=species&column=petal_length"
HTTP/1.1 200 OK
date: Fri, 14 Aug 2026 01:37:51 GMT
server: uvicorn
content-length: 285
content-type: application/json

{
  "by": "species",
  "column": "petal_length",
  "agg": "mean",
  "groups": [
    {
      "species": "setosa",
      "count": 50,
      "mean": 1.462
    },
    {
      "species": "versicolor",
      "count": 50,
      "mean": 4.26
    },
    {
      "species": "virginica",
      "count": 50,
      "mean": 5.552
    }
  ]
}

PS C:\Users\John\data-api> curl.exe -i "http://127.0.0.1:8000/group_by?by=species&column=sepal_length&agg=max"
HTTP/1.1 200 OK
date: Fri, 14 Aug 2026 01:37:51 GMT
server: uvicorn
content-length: 196
content-type: application/json

{
  "by": "species",
  "column": "sepal_length",
  "agg": "max",
  "groups": [
    {
      "species": "setosa",
      "count": 50,
      "max": 5.8
    },
    {
      "species": "versicolor",
      "count": 50,
      "max": 7.0
    },
    {
      "species": "virginica",
      "count": 50,
      "max": 7.9
    }
  ]
}
```

Figure 15: GET /group_by, with mean and then max

The correlation matrix. Petal length and petal width come out at 0.963, nearly collinear, while sepal width is slightly negatively correlated with everything else:

```
> curl - /correlation

PS C:\Users\John\data-api> curl.exe -i http://127.0.0.1:8000/correlation
HTTP/1.1 200 OK
date: Fri, 14 Aug 2026 01:37:51 GMT
server: uvicorn
content-length: 506
content-type: application/json

{
  "columns": [
    "sepal_length",
    "sepal_width",
    "petal_length",
    "petal_width"
  ],
  "matrix": {
    "sepal_length": {
      "sepal_length": 1.0,
      "sepal_width": -0.11757,
      "petal_length": 0.871754,
      "petal_width": 0.817941
    },
    "sepal_width": {
      "sepal_length": -0.11757,
      "sepal_width": 1.0,
      "petal_length": -0.42844,
      "petal_width": -0.366126
    },
    "petal_length": {
      "sepal_length": 0.871754,
      "sepal_width": -0.42844,
      "petal_length": 1.0,
      "petal_width": 0.962865
    },
    "petal_width": {
      "sepal_length": 0.817941,
      "sepal_width": -0.366126,
      "petal_length": 0.962865,
      "petal_width": 1.0
    }
  }
}
```

Figure 16: GET /correlation

The failure cases

A read before anything is loaded, and a column that does not exist. The 404 lists the columns that do exist, which saves the caller a trip to the documentation:

```
> _ curl - 409 and 404

PS C:\Users\John\data-api> curl.exe -i http://127.0.0.1:8000/columns
HTTP/1.1 409 Conflict
date: Fri, 14 Aug 2026 01:37:51 GMT
server: uvicorn
content-length: 89
content-type: application/json

{
  "error": "data_not_loaded",
  "message": "No dataset is loaded. Call POST /load_data first."
}

PS C:\Users\John\data-api> curl.exe -i "http://127.0.0.1:8000/filter_data?column=height&op=gt&value=1"
HTTP/1.1 404 Not Found
date: Fri, 14 Aug 2026 01:37:51 GMT
server: uvicorn
content-length: 136
content-type: application/json

{
  "error": "column_error",
  "message": "No column named 'height'. Available: sepal_length, sepal_width, petal_length, petal_width, species."
}
```

Figure 17: 409 before loading, and 404 for an unknown column

Then three more: text compared against a numeric column, statistics asked for on a text column, and an operator the API does not accept. The last one is a 422 rather than a 400 because FastAPI rejects it from the Literal before the endpoint runs, and its message names the seven values it would have accepted:

```
> curl -400, 404 and 422

PS C:\Users\John\data-api> curl.exe -i "http://127.0.0.1:8000/filter_data?column=petal_length&op=gt&value=big"
HTTP/1.1 400 Bad Request
date: Fri, 14 Aug 2026 01:37:51 GMT
server: uvicorn
content-length: 97
content-type: application/json

{
  "error": "filter_error",
  "message": "'big' is not a number, and column 'petal_length' is float64."
}

PS C:\Users\John\data-api> curl.exe -i http://127.0.0.1:8000/stats/species
HTTP/1.1 404 Not Found
date: Fri, 14 Aug 2026 01:37:51 GMT
server: uvicorn
content-length: 69
content-type: application/json

{
  "error": "column_error",
  "message": "Column 'species' is not numeric."
}

PS C:\Users\John\data-api> curl.exe -i "http://127.0.0.1:8000/filter_data?column=species&op=wat&value=setosa"
HTTP/1.1 422 Unprocessable Entity
date: Fri, 14 Aug 2026 01:37:51 GMT
server: uvicorn
content-length: 219
content-type: application/json

{
  "detail": [
    {
      "type": "literal_error",
      "loc": [
        "query",
        "op"
      ],
      "msg": "Input should be 'eq', 'ne', 'gt', 'gte', 'lt', 'lte' or 'contains'",
      "input": "wat",
      "ctx": {
        "expected": "'eq', 'ne', 'gt', 'gte', 'lt', 'lte' or 'contains'"
      }
    }
  ]
}
```

Figure 18: 400 for a value that cannot be compared, 404 for statistics on text, and FastAPI's own 422 for a bad operator

Step 5: The generated documentation

FastAPI builds an OpenAPI document from the type hints and the response models, and serves two interactive views of it. Nothing here was written by hand; the descriptions come from the endpoint decorators and the field descriptions in schemas.py.

Data Processing API 1.0.0 OAS 3.1

/openapi.json

A small API for poking at a tabular dataset over HTTP.

The dataset ships with the project: the Iris measurements, 150 rows and five columns. [POST /load_data](#) reads it into a Pandas DataFrame that is kept in memory; every other endpoint works on that DataFrame and returns **409** if it has not been loaded yet.

Pandas does the loading, describing, filtering and grouping. NumPy does the statistics in [/stats/{column}](#) and the correlation matrix.

Figure 19: The service description at /docs, taken from the FastAPI constructor

All eight endpoints, grouped by the tags they were given:

data	Loading and inspecting the dataset.	^
POST	/load_data Load the dataset into a DataFrame	▼
GET	/columns The columns, their types and their null counts	▼
GET	/describe_data Summary statistics from pandas describe()	▼
GET	/filter_data Filter rows on a condition	▼
analysis	Statistics over the loaded data.	^
GET	/stats/{column} NumPy statistics for one numeric column	▼
GET	/group_by Group by one column and aggregate another	▼
GET	/correlation Pearson correlation between the numeric columns	▼
service	Service status.	^
GET	/health Is the service up, and is anything loaded	▼

Figure 20: The endpoints at /docs

Expanding POST /load_data shows the response model with every field described, and an example built from the model itself:

POST

/load_data

Load the dataset into a DataFrame

Reads the bundled CSV with `pandas.read_csv`, lowercases the column names and keeps the DataFrame in memory. Calling it again reloads from disk and discards any previous copy.

Parameters

Try it out

No parameters

Responses

Code	Description	Links
200	Successful Response Media type application/json Controls Accept header. Example Value Schema <pre> { "source": "data/iris.csv", "loaded_at": "string", "rows": 150, "columns": 5, "column_names": ["string"], "dtypes": { "additionalProp1": "string", "additionalProp2": "string", "additionalProp3": "string" }, "memory_bytes": 0, "preview": [{ "additionalProp1": {} }] } </pre>	No links
404	No such column. Media type application/json Example Value Schema <pre> { "error": "data_not_loaded", "message": "No dataset is loaded. Call POST /load_data first." } </pre>	No links

Figure 21: POST /load_data expanded

/filter_data shows what the type hints bought. Each parameter has its description and example, op is a dropdown of the seven operators, limit shows its range, and the three error responses are listed with the shape they return:

GET

/filter_data

Filter rows on a condition

Builds a boolean mask and returns the rows that match. The value arrives as text and is converted to a number when the column is numeric, so `value=5` and `value=5.0` behave the same on a float column.

Parameters

Try it out

Name	Description
column ★ required	Column to test.
string (query)	column
op	Comparison to apply.
string (query)	Available values : eq, ne, gt, gte, lt, lte, contains Default value : eq
	eq
value ★ required	Value to compare against.
string (query)	value
limit	Most rows to return.
integer (query)	Default value : 10
	10 maximum: 150 minimum: 1

Responses

Figure 22: /filter_data expanded, with the parameters FastAPI derived from the function signature

The schemas section lists every model. ValidationError and HTTPValidationError were not written by me; FastAPI adds them because it does the parameter validation:

Schemas			^
ColumnInfo	>	Expand all	object
ColumnsResult	>	Expand all	object
CorrelationResult	>	Expand all	object
DescribeResult	>	Expand all	object
Error	>	Expand all	object
FilterResult	>	Expand all	object
GroupByResult	>	Expand all	object
HTTPValidationError	>	Expand all	object
Health	>	Expand all	object
LoadResult	>	Expand all	object
OutlierBounds	>	Expand all	object
StatsResult	>	Expand all	object
ValidationError	>	Expand all	object

Figure 23: The generated schemas

StatsResult expanded, with the descriptions from the Pydantic fields:

StatsResult ^	Collapse all	object
Computed with NumPy rather than pandas.		
column*	>	string
count*	>	integer
mean*	>	Expand all (number null)
median*	>	Expand all (number null)
std	>	Expand all (number null)
variance*	>	Expand all (number null)
min*	>	Expand all (number null)
max*	>	Expand all (number null)
range*	>	Expand all (number null)
q1*	>	Expand all (number null)
q3*	>	Expand all (number null)
iqr*	>	Expand all (number null)
outlier_bounds*	>	Expand all object
outliers*	>	Expand all array<number>
ValidationError	>	Expand all object

Figure 24: The StatsResult schema

The same document also renders as ReDoc at /redoc, which is easier to read end to end:

Search...

data >

analysis >

service >

Data Processing API (1.0.0)

Download OpenAPI specification: [Download](#)

A small API for poking at a tabular dataset over HTTP.

The dataset ships with the project: the `Iris` measurements, 150 rows and five columns. `POST /load_data` reads it into a Pandas DataFrame that is kept in memory; every other endpoint works on that DataFrame and returns `409` if it has not been loaded yet.

Pandas does the loading, describing, filtering and grouping. NumPy does the statistics in `/stats/{column}` and the correlation matrix.

data

Loading and inspecting the dataset.

Load the dataset into a DataFrame

Reads the bundled CSV with `pandas.read_csv`, lowercases the column names and keeps the DataFrame in memory. Calling it again reloads from disk and discards any previous copy.

Responses

> 200 Successful Response

> 404 No such column.

POST /load_data

Response samples

200 404

Content type: application/json

Copy Expand all Collapse all

```
{
  "source": "data/iris.csv",
  "loaded_at": "string",
  "rows": 150,
  "columns": 5,
  "column_names": [
    "string"
  ],
  "dtypes": {
    "property1": "string",
```

Figure 25: The same API at /redoc

Calling the API from the documentation

Swagger UI can call the service itself, which makes the documentation a test client. The responses below came back from the running API.

Loading the data from the browser. This has to happen first, for the same reason it does from curl:

POST

/load_data

Load the dataset into a DataFrame

⌵

Reads the bundled CSV with `pandas.read_csv`, lowercases the column names and keeps the DataFrame in memory. Calling it again reloads from disk and discards any previous copy.

Parameters

Cancel

No parameters

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
'http://127.0.0.1:8000/load_data' \
-H 'accept: application/json' \
-d ''
```

Request URL

```
http://127.0.0.1:8000/load_data
```

Server response

Code

Details

200

Response body

```
{
  "source": "data/iris.csv",
  "loaded_at": "2026-08-14T01:36:47+00:00",
  "rows": 150,
  "columns": 5,
  "column_names": [
    "sepal_length",
    "sepal_width",
    "petal_length",
    "petal_width",
    "species"
  ],
  "dtypes": {
    "sepal_length": "float64",
    "sepal_width": "float64",
    "petal_length": "float64",
    "petal_width": "float64",
    "species": "object"
  },
  "memory_bytes": 14728,
  "preview": [
    {
      "sepal_length": 5.1,
      "sepal_width": 3.5,
      "petal_length": 1.4,
      "petal_width": 0.2,
      "species": "setosa"
    }
  ],
}
```

Download

Response headers

```
content-length: 810
content-type: application/json
date: Fri, 14 Aug 2026 01:36:46 GMT
server: uvicorn
```

Figure 26: POST /load_data executed from /docs

Then filtering, with the parameters filled in the form. The curl command Swagger UI shows is the same request written out, which is a convenient way to get a working command line:

GET
/filter_data
Filter rows on a condition

Builds a boolean mask and returns the rows that match. The value arrives as text and is converted to a number when the column is numeric, so `value=5` and `value=5.0` behave the same on a float column.

Parameters
Cancel

Name	Description
column required string (query)	Column to test. petal_length
op string (query)	Comparison to apply. eq
value required string (query)	Value to compare against. 5.0
limit integer (query)	Most rows to return. 3 maximum: 150 minimum: 1

Execute
Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/filter_data?column=petal_length&op=eq&value=5.0&limit=3' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/filter_data?column=petal_length&op=eq&value=5.0&limit=3
```

Server response

Code
Details

200

Response body

```
{
  "column": "petal_length",
  "op": "eq",
  "value": "5.0",
  "matched": 4,
  "total": 150,
  "returned": 3,
  "limit": 3,
  "rows": [
    {
      "sepal_length": 6.7,
      "sepal_width": 3,
      "petal_length": 5,
      "petal_width": 1.7,
      "species": "versicolor"
    },
    {
      "sepal_length": 5.7,
      "sepal_width": 2.5,
      "petal_length": 5,
      "petal_width": 2,
      "species": "virginica"
    },
    {
      "sepal_length": 6,
      "sepal_width": 2.2,
      "petal_length": 5,
      "petal_width": 1.5,

```

Response headers

```
content-length: 400
content-type: application/json
date: Fri, 14 Aug 2026 01:36:50 GMT
server: uvicorn
```

Figure 27: `/filter_data` executed from `/docs`, with the equivalent `curl` command

And the statistics endpoint, with the column supplied as a path parameter:

GET

/stats/{column}

NumPy statistics for one numeric column

⌵

Mean, median, sample standard deviation, variance, quartiles and the Tukey outlier fences, all computed with NumPy on the column as a float array.

Parameters

Cancel

Name	Description
column required	A numeric column.
string (path)	petal_length

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/stats/petal_length' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/stats/petal_length
```

Server response

Code	Details
200	Response body <pre>{ "column": "petal_length", "count": 150, "mean": 3.758, "median": 4.35, "std": 1.765298, "variance": 3.116278, "min": 1, "max": 6.9, "range": 5.9, "q1": 1.6, "q3": 5.1, "iqr": 3.5, "outlier_bounds": { "lower": -3.65, "upper": 10.35 }, "outliers": [] }</pre> Copy Download Response headers <pre>content-length: 220 content-type: application/json date: Fri, 14 Aug 2026 01:36:53 GMT server: uvicorn</pre>

Figure 28: /stats/petal_length executed from /docs

Automated tests

40 tests cover the endpoints. They are worth more than the screenshots, because a screenshot proves the API worked once and these keep proving it.

```
>_ Windows PowerShell - pytest

PS C:\Users\John\data-api> python -m pytest tests/ -q
..... [100%]
40 passed in 0.33s

PS C:\Users\John\data-api> python -m pytest tests/ -v -k "filter or stats or correlation"
test_filter_equals_on_text PASSED [ 5%]
test_filter_greater_than_on_a_number PASSED [ 10%]
test_filter_value_is_coerced_for_numeric_columns PASSED [ 15%]
test_filter_operators PASSED [ 21%]
test_filter_contains_is_case_insensitive PASSED [ 26%]
test_filter_limit_caps_the_rows_but_not_the_count PASSED [ 31%]
test_filter_with_no_matches_is_still_a_200 PASSED [ 36%]
test_filter_unknown_column_is_404 PASSED [ 42%]
test_filter_unknown_operator_is_422 PASSED [ 47%]
test_filter_non_numeric_value_on_numeric_column_is_400 PASSED [ 52%]
test_filter_contains_on_a_numeric_column_is_400 PASSED [ 57%]
test_filter_limit_is_validated PASSED [ 63%]
test_stats_matches_known_values PASSED [ 68%]
test_stats_quartiles_and_iqr_agree PASSED [ 73%]
test_stats_outlier_bounds_are_tukey_fences PASSED [ 78%]
test_stats_unknown_column_is_404 PASSED [ 84%]
test_stats_on_a_text_column_is_404 PASSED [ 89%]
test_correlation_shape_and_diagonal PASSED [ 94%]
test_correlation_is_symmetric_and_known PASSED [100%]
===== 19 passed, 21 deselected in 0.15s =====
```

Figure 29: The test run, and the filter, statistics and correlation tests by name

The values are asserted against numbers that are known for this dataset: a petal length mean of 3.758, a median of 4.35, 42 rows above 5.0, a setosa mean of 1.462 and a petal length to petal width correlation of 0.963. If a filter or a statistic quietly broke, those would catch it, which a test that only checks for a 200 would not.

Several tests exist because of mistakes I made. One checks that no response body contains the text NaN, after describe() first failed to encode. One checks that value=5 and value=5.0 match the same 42 rows, after an early version compared the query string against a float column as text and returned nothing. One checks that every read returns 409 before loading, across all six read endpoints at once.

What I would change

The DataFrame in memory is the obvious weakness. It is a module-level variable, so two workers would each have their own copy and a client would get different answers depending on which one it reached. For anything real the data would be loaded at startup, or held somewhere both workers could see.

I would also add pagination properly. /filter_data has a limit but no offset, so there is no way to walk past the first page of matches. The matched and returned counts in the response were meant to make the truncation obvious, and they do, but they are not a substitute for being able to ask for

the rest.

What went better than I expected was how much of the documentation came for free. Declaring op as a Literal, giving limit a range and putting descriptions on the Pydantic fields produced the parameter table, the dropdown, the validation and the examples in the screenshots above without any separate specification to keep in step. On the previous assignment I wrote an OpenAPI document by hand and needed tests to stop it drifting from the code; here the code is the document.